



# Informe de justificación técnica

Evolución de frase objetivo

Docente:

*Mgter. Walter Fabian Agüero*

Estudiante/s:

*Orellana Galo*

---

|                                                                      |          |
|----------------------------------------------------------------------|----------|
| <b>1. Introduccion.....</b>                                          | <b>3</b> |
| <b>2. Representación del genotipo y definición del problema.....</b> | <b>3</b> |
| <b>3. Función de Aptitud.....</b>                                    | <b>3</b> |
| <b>4. Operadores evolutivos y configuracion de parametros.....</b>   | <b>4</b> |
| 4.1. Metodo de seleccion: Seleccion por torneo vs Ruleta.....        | 4        |
| 4.2. Crossover, mutación y elitismo.....                             | 4        |
| <b>5. Visualización grafica y resultados de ejecución.....</b>       | <b>5</b> |
| <b>6. Declaración de Vibe Coding y asistencia de IA.....</b>         | <b>6</b> |
| <b>7. Conclusiones.....</b>                                          | <b>6</b> |

## 1. Introducción

Los Algoritmos Genéticos (AG) son métodos computacionales de búsqueda y optimización inspirados en los principios de la evolución biológica y la selección natural postulado por Charles Darwin. Estos algoritmos operan sobre una población de soluciones potenciales (representadas mediante cromosomas) y aplican iterativamente operadores evolutivos de evaluación de aptitud, selección, crossover y mutación para guiar la búsqueda hacia soluciones ideales en espacios de búsqueda complejos.

En el marco de la materia de Inteligencia Artificial de la Universidad Nacional de Río Negro, este trabajo práctico tiene como objetivo implementar un algoritmo genético completo en lenguaje Python, justificar sus decisiones de diseño técnico, analizar su convergencia y proporcionar una representación visual dinámica que permita ver cuadro a cuadro la adaptación y evolución de la población. Además se agregará una evaluación estadística.

## 2. Representación del genotipo y definición del problema

Para esta implementación se seleccionó el problema de la 'Evolución de Frase Objetivo'. El objetivo computacional es hacer aparecer, a partir de cadenas de texto generadas en su totalidad al azar, la frase que nosotros definamos, en este caso: 'INTELIGENCIA ARTIFICIAL'.

Las especificaciones de la representación genética son las que muestro a continuación:

- Cromosoma (Individuo): Longitud  $N = 23$  caracteres (correspondientes a los 23 espacios y letras de la palabra objetivo).
- Espacio Genético (Genes): Abecedario extendido en español compuesto por 35 caracteres posibles: A - Z (mayúsculas), Ñ, las vocales acentuadas y el espacio en blanco.
- Tamaño del espacio de búsqueda:  $N = 35^{23} = 3.92 \cdot 10^{35}$  combinaciones posibles, lo que demuestra la necesidad de una búsqueda heurística evolutiva eficiente en lugar de una búsqueda por fuerza bruta.

## 3. Función de Aptitud

La función de aptitud o "fitness" es la métrica matemática encargada de cuantificar la calidad o adaptación de cada individuo respecto al objetivo planteado. Para este problema se definió una función de coincidencia posicional exacta normalizada:

**fitness (individuo) = ( suma de coincidencias posicionales de caracteres ) / (longitud de la frase)**

Algunas propiedades de la función de aptitud:

- El valor esta acotado en el intervalo de 0.0 a 1.0.
- Un fitness de 0.0 indica 0% de coincidencia (ninguna letra correcta en su posición de la frase objetivo).
- Un fitness de 1.0 indica un 100% de coincidencia exacta con la frase objetivo.
- Proporciona una optimización gradual suave, permitiendo así a los métodos de selección identificar mejoras de a poco.

## 4. Operadores evolutivos y configuración de parametros

El éxito y la velocidad de convergencia de un algoritmo genético dependen del equilibrio entre la exploración, es decir, mantener la diversidad en la población y de la explotación al seleccionar las mejores soluciones.

### 4.1. Metodo de seleccion: Seleccion por torneo vs Ruleta

En el script desarrollado, se implemento de forma primaria la selección por torneo ( $k = 5$ ), dejando además disponible la función de selección por ruleta para analisis comparativo:

Selección por torneo (la que elegí): En cada iteración se eligen  $k = 5$  individuos al azar de la población y el individuo con la mayor aptitud gana el derecho a ser progenitor. Este método ofrece una presión selectiva constante, se puede ajustar fácilmente el tamaño del torneo  $k$ , y evita la dominancia prematura de un super individuo en las primeras generaciones.

Selección por ruleta (opción secundaria): Cada individuo recibe una porción de la ruleta proporcional a su fitness. Si bien es el método mas “clásico”, puede provocar un estancamiento si un individuo inicial tiene un fitness significativamente superior al promedio de la población.

### 4.2. Crossover, mutación y elitismo

Crossover: Se elige aleatoriamente un punto de corte entre 1 y  $N-1$ , intercambiando las cadenas genéticas de los dos padres seleccionados.

Mutación: Tasa de mutación  $P_m = 0.01$  (1% por gen). Cada caracter de la cadena descendiente tiene un 1% de probabilidad de ser reemplazado por un nuevo caracter aleatorio del abecedario. La mutación es importante para introducir nueva variabilidad genética y prevenir la traba o el freno en óptimos locales.

Elitismo: Conservación del mejor individuo. En cada paso generacional, el individuo de mayor fitness se copia intacto a la nueva generación sin pasar por mutación. Esto garantiza que la maxima aptitud alcanzada sea estrictamente no decreciente al paso del tiempo.

Cuadro de parametro del algoritmo:

| Parametro               | Valor configurado          | Justificacion                                                         |
|-------------------------|----------------------------|-----------------------------------------------------------------------|
| N (tamaño de poblacion) | 150                        | Da suficiente diversidad inicialmente                                 |
| P_m (Tasa de mutacion)  | 0.01 (1%)                  | Balance entre mantener la memoria genética y explorar nuevos genes    |
| K (tamaño de torneo)    | 5                          | Presión selectiva alta que acelera la convergencia                    |
| Estrategia de reemplazo | Elitista + Generacional    | El mejor pasa directo, el resto se renueva mediante selección o cruce |
| Criterio de freno       | Fitness = 1.0 o Gen = 1000 | Se para automáticamente al llegar a la coincidencia de 100%           |

## 5. Visualización grafica y resultados de ejecución

En cumplimiento con el requisito establecido por la catedra, el script de python incluye una interfaz grafica interactiva desarrollada con matplotlib en modo dinámico. Durante la ejecución, la pantalla despliega dos curvas continuas en tiempo real.

En una traza de color azul se ve el progreso del mejor individuo de la poblacion. Por otro lado, con una linea de color naranja se ve el fitness medio de toda la poblacion, mostramo como el grupo entero evoluciona y aumenta gradualmente hacia la solucion. Por ultimo, hay un cuadro que muestra la generación actual, la mejor cadena de texto alcanzada y el porcentaje de coincidencia exacta.

Para evaluar el comportamiento del algoritmo desde una perspectiva estadística, se incorporo al codigo la función de “*evaluar\_rendimiento*”. Esta función ejecuta el ciclo evolutivo completo durante un numero n de iteraciones independientes, registrando el numero de generaciones requeridas para converger, la tasa de exito y los valores extremos.

Se ejecuto la prueba con n = 1000 ejecuciones. Los resultados cuantitativos obtenidos se resumen en las siguientes métricas:

| Metrica                              | Resultado obtenido |
|--------------------------------------|--------------------|
| Ejecuciones totales                  | 1000 iteraciones   |
| Promedio de generaciones a resultado | 82.03 generaciones |

|                                   |                  |
|-----------------------------------|------------------|
| Tasa de exito                     | 100% (1000/1000) |
| Minimo de generaciones requeridas | 31 generaciones  |
| Maximo de generaciones requeridas | 233 generaciones |

El analisis de las 1000 ejecuciones permite extraer las siguientes conclusiones técnicas:

- En el 100% de las iteraciones, el algoritmo genético encontro de manera exacta la frase objetivo. Esto demuestra que la combinación de selección por torneo, mutación y elitismo previene totalmente el estancamiento permanente en óptimos locales.
- Un promedio de 82 generaciones implica evaluar unicamente 12300 individuos en promedio. Comparando contra las  $3.92 * 10^{35}$  combinaciones del espacio de búsqueda total, el algoritmo reduce la complejidad computacional en un orden enorme de magnitud, demostrando la potencia de la búsqueda heurística evolutiva.
- La brecha entre el caso óptimo y el caso desfavorable refleja la naturaleza estocástica del azar biológico simulado. En el mejor de los casos, las mutaciones iniciales acertaron genes criticos tempranamente. En el peor de los casos, la población requirió mas iteraciones de cruzamiento para reensamblar esquemas genéticos favorables, sin embargo, incluso en su peor registro, la respuesta se obtuvo rápidamente.

## 6. Declaración de Vibe Coding y asistencia de IA

Conforme a las pautas establecidas por el docente y las normativas académicas descritas en el apunte de la catedra, se declara abiertamente el uso de 'Vibe Coding' para el desarrollo del presente trabajo practico.

El uso principal de esta técnica/herramienta esta en la asistencia para la creación y estructuración inicial de la animación en Matplotlib mediante prompts en lenguaje natural.

A pesar del uso de la herramienta, como autor de este informe he validado la lógica, el codigo y los cálculos del ya mencionado para garantizar una ejecución estable y sin errores.

## 7. Conclusiones

La realización de este trabajo practico permitió consolidar de forma practica y concreta los conceptos teóricos de la computación evolutiva. Se demostro como los principios de selección, cruce y mutación logran resolver un problema de optimización en un espacio de  $3.92 * 10^{35}$  combinaciones posibles en apenas un centenar de iteraciones. El analisis estadístico sobre 1000 ejecuciones justifica de forma técnica los parámetros elegidos, garantizando una convergencia del 100% en un promedio de 82 generaciones.